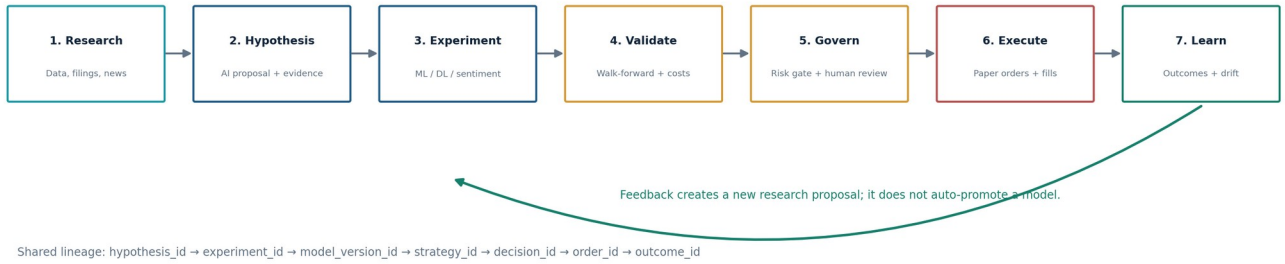
FINSIGHT

# RESEARCHOPS

## Complete Production-Engineering Project Plan

AI-assisted financial research, quantitative experimentation, ML/DL, reinforcement learning, risk governance and paper-execution feedback.

**End-to-end working application flow**




*The core research-to-outcome lifecycle*

**Architecture • GitHub references • Data design • AI/ML/RL • APIs • Security • Testing • Deployment**

Prepared: 2 November 2026 | Intended scope: individual-developer portfolio product with production-engineering evidence

# Document guide

This document consolidates the complete 26-section implementation plan discussed for FinSight ResearchOps. It is a blueprint, not a claim that the application has already been implemented or deployed. Repository references are provided for learning and selective reuse; upstream code must be checked for licence, dependency and security compatibility before incorporation.

**Important product boundary**
The initial release is a production-engineered portfolio application for research and paper simulation. It is not a regulated financial institution platform, investment adviser, or real-money trading system. No live broker execution is included in the initial scope.

# Contents - all 26 sections


- 1. Product definition
- 2. The gap the platform fills
- 3. GitHub repositories and reuse strategy
- 4. System design and architecture
- 5. Complete technical stack
- 6. Data engineering and data-source design
- 7. Database design
- 8. AI and research-engine architecture
- 9. ML, deep learning and sentiment engineering
- 10. Quantitative backtesting and strategy engineering
- 11. Reinforcement learning portfolio management
- 12. Model governance and promotion
- 13. Paper execution and operational risk
- 14. APIs and application contracts
- 15. Application folder structure
- 16. Security and permissions
- 17. Testing and quality strategy
- 18. CI/CD and deployment architecture
- 19. Frontend pages and working application flow
- 20. Implementation roadmap and acceptance criteria
- 21. Project configuration and developer experience
- 22. Production-readiness checklist
- 23. Interview demonstration plan
- 24. Definition of success
- 25. Resume positioning
- 26. Final implementation recommendation

# 1. Product definition

Product name: FinSight ResearchOps. Product category: AI-powered financial research and quantitative strategy operations platform. Its purpose is to manage the lifecycle from a financial hypothesis through research, reproducible ML/DL experiments, backtesting, risk approval, paper execution and measured outcomes.

## Problem statement

Financial research, model development, backtesting, strategy approval and execution are frequently handled in different tools. This separation makes it difficult to reconstruct why a strategy was created, which data it used, which model generated a decision, whether the decision passed risk review, what orders were submitted and what happened afterwards. ResearchOps makes those relationships explicit and auditable.

## Primary users

| User | Responsibilities |
|---|---|
| Research Analyst | Research companies, documents, financial events and hypotheses. |
| Quant Researcher | Create features, train models and test strategies. |
| ML Engineer | Manage experiments, model versions, evaluation and serving. |
| Risk Reviewer | Review backtests, exposure, drawdown and promotion requests. |
| Portfolio Analyst | Monitor paper portfolios, costs, positions and performance. |
| Platform Admin | Manage identities, permissions, jobs, providers and system health. |

## Initial product boundaries

- US-listed equities and ETFs first; other exchanges can be added through provider adapters.
- Historical daily market data and public financial documents / SEC filings.
- Research, model experimentation, backtesting and paper execution.
- No real-money trading route in the first release.
- No automated lending, credit decisions or investment advice.
- NSE/India market coverage is a later extension, subject to suitable data rights and provider access.

**Product promise**
Every strategy decision can be traced back to its research hypothesis, data snapshot, feature schema, model version, backtest, risk approval, order/fill and measured outcome.

# 2. The gap the platform fills

The platform should not be positioned as another stock-price predictor or AI chatbot. Its differentiator is a governed, closed-loop research-to-outcome lifecycle.

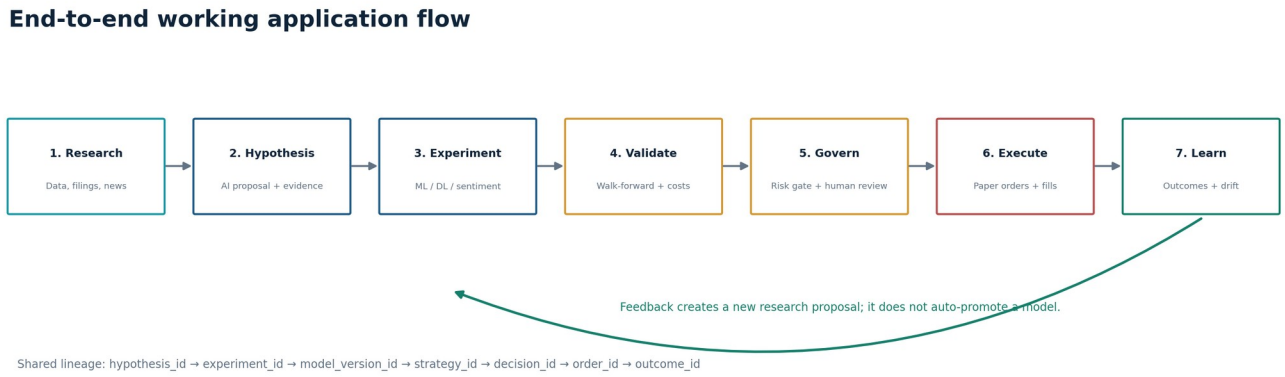



*Figure 1. End-to-end application workflow. Feedback starts a new research cycle but does not automatically promote a model.*

## Observed gaps in existing project patterns

| Existing pattern / limitation | ResearchOps capability |
|---|---|
| Research reports are disconnected from model experiments. | Every research hypothesis has a unique ID and is linked to experiments. |
| Backtests may use inconsistent or future information. | Point-in-time data snapshots and temporal-integrity tests. |
| Models can be treated as ready after training alone. | Model registry, evaluation gates and human promotion approval. |
| Paper orders are separate from model decisions. | Shared strategy, decision, order and execution ledger. |
| Execution outcomes are not systematically fed into research. | Prediction-versus-outcome evaluation and evidence-based follow-up experiments. |

## Reference observations and opportunity

The Stock Agent System repository documents separate analysis and trading pipelines, and notes limitations around historical news reconstruction, same-day-close backtest timing and pending-order handling. The Quant Strategy Framework publishes reproducible backtest results that underperform buy-and-hold for its supplied strategy. Those examples reinforce that an application needs more than a compelling agent output: it needs temporal correctness, realistic fills, benchmark comparisons and ongoing measurement.

Original contribution: the Research-to-Outcome Ledger. It connects hypothesis_id, experiment_id, model_version_id, strategy_id, decision_id, order_id, fill_id and outcome_id into one lineage.

# 3. GitHub repositories and reuse strategy

Use repositories as implementation references and selective sources of code only after inspecting source files, dependencies, licences, maintenance activity and compatibility. Do not merge complete applications wholesale. Build the common ResearchOps domain layer independently.

## Primary implementation references

| Repository | What to study / reuse | Decision and licence note |
|---|---|---|
| Quant Strategy Framework https://github.com/niharmaddina/quant-strategy-framework | Frozen-universe research, indicators, backtesting organization, regression tests. | Primary reference. MIT licence file was verified. Do not assume its supplied strategy is profitable. |
| RL Portfolio Management https://github.com/MustafaBeratYavas/rl-based-algorithmic-trading-and-portfolio-management | Gymnasium environment, PPO/SAC workflow, config-driven data preparation, evaluation and Docker/testing patterns. | Primary reference. MIT licence file was verified. |
| Financial Research Agent https://github.com/Harshithaaramesh/financial-research-agent | LangGraph analyst workflow, SEC document ingestion, RAG and research report structure. | Primary reference. MIT licence file was verified. |
| Stock Agent System https://github.com/mustafagoktugibolar/stock-agent-system | Analyst/trading separation, caching, paper trading, reflection and portfolio-state patterns. | Architecture reference. README has MIT metadata; verify repository-level licence before code reuse. |
| Banking Fraud & Credit Risk ML https://github.com/Speaking-Data-NK/banking-fraud-risk-ml | MLflow, FastAPI model serving, PostgreSQL persistence, CI, Docker and monitoring. | Optional engineering reference. MIT licence file verified. Fraud/credit are not core product features. |

## Additional repositories for study

- AI/ML Stock Market Prediction: https://github.com/AISHDM/stock-market-prediction - XGBoost, LSTM, sentiment fusion and honest baseline comparisons. Verify licence before copying code.
- RLTradingAgent: https://github.com/maksimprivalov/RLTradingAgent - small PPO/Gymnasium example useful for reward-design study; no licence file was verified.
- Machine Learning Alpha Model: https://github.com/joel-cdev/machine-learning-alpha-model - Random Forest, XGBoost, LSTM and Streamlit reference; verify licence before reuse.

## Large projects deliberately excluded from core dependencies

The following are useful for reading designs, but should not be mandatory dependencies in the initial product: Microsoft Qlib, Microsoft RD-Agent, AI4Finance FinRL-X, FinRobot and FinGPT, TauricResearch TradingAgents, and OpenBB. Reimplement only the functions needed behind narrow interfaces. This reduces dependency complexity and improves explainability.

| Large repository | Reference URL | Use only as inspiration for |
|---|---|---|
| Microsoft Qlib | https://github.com/microsoft/qlib | Quant research concepts and model catalog. |
| Microsoft RD-Agent | https://github.com/microsoft/RD-Agent | Autonomous factor and model research ideas. |
| FinRL-X | https://github.com/AI4Finance-Foundation/FinRL-Trading | Modular portfolio allocation and risk overlays. |
| FinRobot | https://github.com/AI4Finance-Foundation/FinRobot | Financial deterministic calculations and report design. |
| FinGPT | https://github.com/AI4Finance-Foundation/FinGPT | Financial NLP and model-training concepts; inspect model-weight licences. |
| TradingAgents | https://github.com/TauricResearch/TradingAgents | Multi-agent analyst organization. |

| Large repository | Reference URL | Use only as inspiration for |
|---|---|---|
| OpenBB | https://github.com/openbq-org/OpenBB | Financial data integration concepts; check data terms. |

DocuMind note: the previously referenced Vishal93727/documind-agentic-rag repository could not be independently verified through the connected repository lookup. Local DocuMind source may be evaluated for the RAG module after checking origin, dependencies and licence. It is not a prerequisite for this design.

# 4. System design and architecture

Recommended architecture: a modular monolith for domain logic, with independent background workers for research, ML training, RL training, simulation and execution. This provides clear module boundaries without premature microservices.

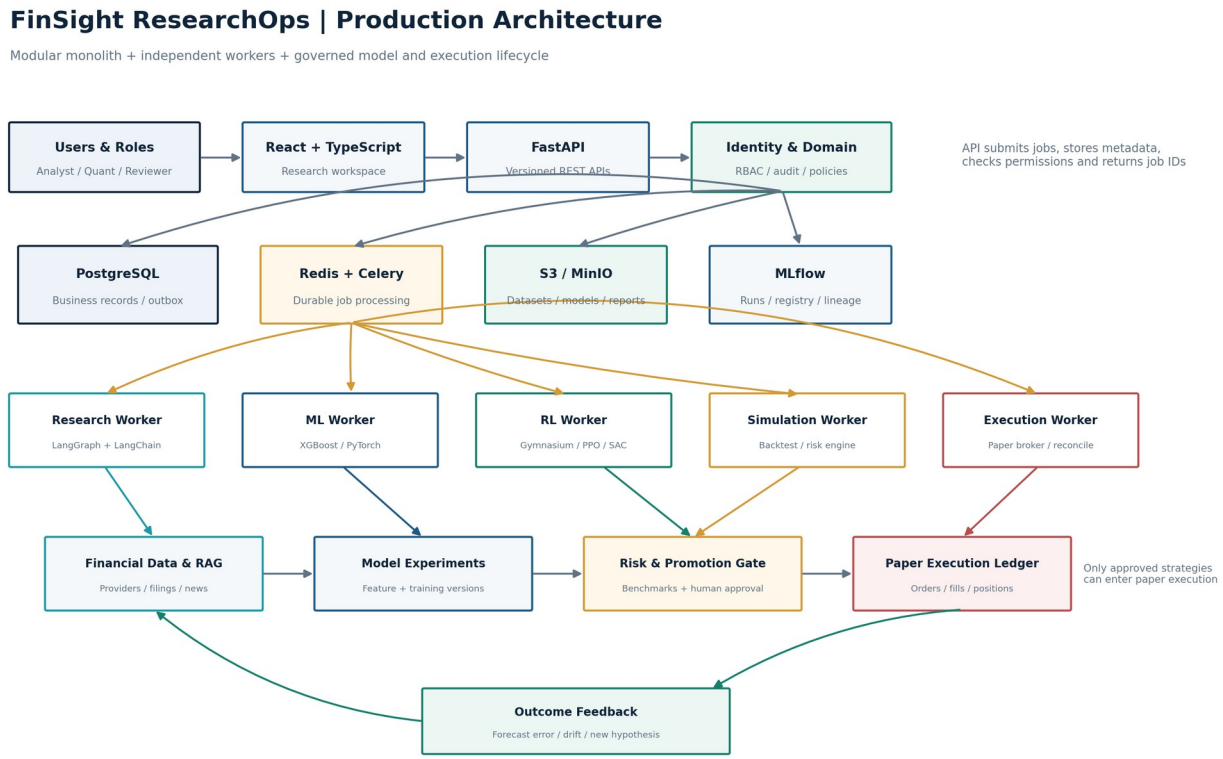



*Figure 2. Logical production architecture. API requests create jobs; background workers execute long-running tasks.*

## Component responsibilities

| Component | Owns |
|---|---|
| Identity | Users, roles, resource permissions and authentication. |
| Financial data | Market data, filings, news, snapshots and provenance. |
| Research | Hypotheses, documents, agent runs, evidence and reports. |
| Feature engineering | Feature definitions, labels, versions and datasets. |
| Experiment manager | Configurations, job state, metrics and reproducibility records. |

| Component | Owns |
|---|---|
| ML/DL platform | Training, evaluation, inference artifacts and model lineage. |
| RL platform | Environment, policy, reward configuration and checkpoints. |
| Backtesting | Portfolio accounting, simulated orders, fills, costs and benchmarks. |
| Risk engine | Exposure limits, drawdown rules and promotion checks. |
| Model governance | Versions, approval decisions, promotion and rollback history. |
| Execution | Paper orders, fills, positions, broker reconciliation and kill switch. |
| Feedback | Prediction outcomes, drift events and follow-up experiment proposals. |
| Analytics | SQL views, operational reports and performance dashboards. |

### Architecture rules

- API routes call application services, not raw SQL or model internals.
- Domain logic must not import FastAPI or frontend code.
- Training and long-running agent execution run in workers, not in HTTP request handlers.
- External market, LLM and broker providers are hidden behind adapters.
- Model and strategy configurations are validated and versioned.
- Important actions emit audit events and structured logs.
- The risk engine is independent of LLM outputs and cannot be overridden by an AI agent.

# 5. Complete technical stack

| Layer | Technology | Reason |
|---|---|---|
| Frontend | React, TypeScript, Vite | Maintainable analyst dashboard and typed API client. |
| API | FastAPI, Pydantic | Versioned REST APIs and input/output validation. |
| ORM / migration | SQLAlchemy, Alembic | Typed data access and controlled schema evolution. |
| Database | PostgreSQL | Transactional source of truth. |
| Queue / cache | Redis, Celery | Asynchronous job dispatch and background processing. |
| Artifact storage | MinIO / S3 | Dataset snapshots, model artifacts and reports. |
| Research storage | Parquet, DuckDB | Efficient historical analytical workloads. |
| RAG | LangChain, pgvector, PostgreSQL full-text search | Hybrid retrieval and document evidence. |

| Layer | Technology | Reason |
|---|---|---|
| Agent orchestration | LangGraph | Stateful research graph, checkpoints and human approval. |
| ML | scikit-learn, XGBoost, LightGBM | Baselines and tabular models. |
| Deep learning | PyTorch | LSTM and Transformer experiments. |
| Financial NLP | FinBERT or another licensed financial checkpoint | News and document sentiment. |
| RL | Gymnasium, Stable-Baselines3 | Custom environments and PPO/SAC policies. |
| Backtesting | Custom event-driven simulator; VectorBT for research | Separate exploratory evaluation from execution simulation. |
| MLOps | MLflow | Experiment tracking, model versions and lineage. |
| Observability | OpenTelemetry, Prometheus, Grafana | Traces, metrics, logs and alerting. |
| CI/CD | GitHub Actions | Automated quality gates and deployments. |
| Security | JWT/OIDC, RBAC, secret manager | Identity, access control and secret protection. |
| Deployment | Docker, Docker Compose, cloud containers | Repeatable local and staged environments. |
| Analytics | SQL, DuckDB, Power BI optional | Business and research reporting. |

**Architecture decision**
Start with one PostgreSQL database and well-defined domain modules. Add separate services or databases only when load, security or operational boundaries justify the additional complexity.

# 6. Data engineering and data-source design

Data quality and point-in-time correctness are foundational. A sophisticated model trained on future or incorrectly timestamped information is not a reliable model.

## Initial data sources

| Data | Candidate source | Use |
|---|---|---|
| Historical OHLCV | Alpaca market-data API or another permitted provider | Daily price and volume history. |
| Development exploration | yfinance | Exploration only, subject to terms and availability. |
| Company filings | SEC EDGAR | 10-K, 10-Q, 8-K and XBRL facts. |
| News | Licensed news API or permitted RSS source | News event research and sentiment. |
| Benchmark | SPY or configured index/ETF | Benchmark-relative evaluation. |

| Data | Candidate source | Use |
|---|---|---|
| Paper execution | Alpaca paper account adapter | Simulated broker orders. |

SEC submissions and company-facts endpoints provide public JSON data. Use an identifying User-Agent and follow SEC fair-access policies. Market-data provider terms differ: public website access does not automatically grant commercial redistribution rights. Keep provider choice configurable.

## Data pipeline

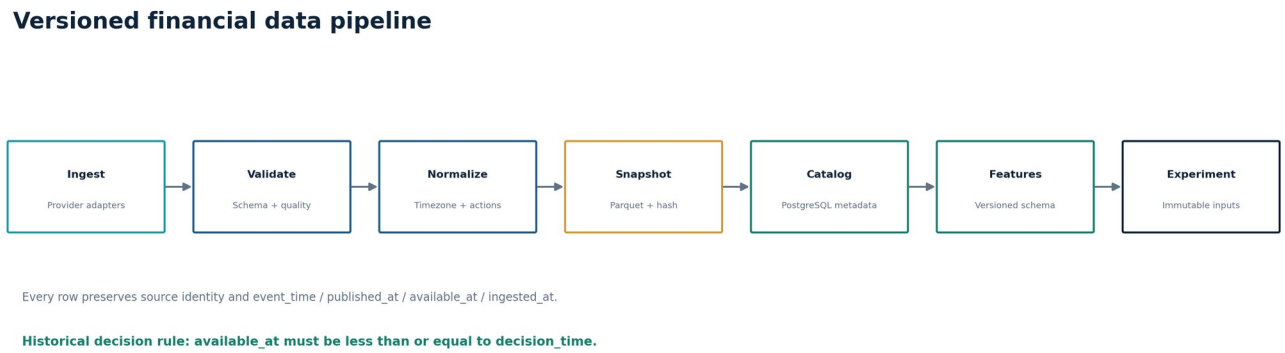



*Figure 2A. Each snapshot is versioned and carries point-in-time availability metadata.*

```
Scheduled ingestion job
  -> provider adapter
  -> schema validation
  -> timezone and corporate-action normalization
  -> data-quality checks
  -> immutable Parquet snapshot
  -> manifest and SHA-256 hash
  -> PostgreSQL data catalog
  -> feature engineering
  -> versioned feature dataset
  -> experiment manager
```

## Time metadata

| Field | Meaning |
|---|---|
| event_time | When the economic or market event occurred. |
| published_at | When the source published the information. |
| available_at | When the system could legitimately use it. |
| ingested_at | When the application collected it. |
| source_id | Provider and source record identity. |

Historical experiments must enforce available_at <= decision_time. Store publication and availability times separately; do not infer availability from ingestion time alone.

### Example snapshot manifest

```
{
  "snapshot_id": "snapshot-2026-001",
  "provider": "example-market-provider",
  "universe_version": "us-etf-universe-v1",
  "start_date": "2023-01-01",
  "end_date": "2025-12-31",
  "interval": "1d",
  "timezone": "UTC",
  "adjustment_policy": "split_and_dividend_adjusted",
  "schema_version": "1.0",
  "row_count": 12000,
  "sha256": "computed-file-hash"
}
```

The manifest is illustrative. Actual row counts, dates, provider and hash must be computed from the ingested dataset.

# 7. Database design

PostgreSQL is the source of truth for business records. Large data files and trained model binaries are stored in object storage, with their metadata and hashes in PostgreSQL.

| Table | Important fields / responsibility |
|---|---|
| users | id, email, password_hash, status. |
| roles / user_roles | Role catalogue and assignments. |
| instruments | symbol, exchange, asset_type. |
| data_sources | provider, configuration reference. |
| data_snapshots | manifest, hash, status, temporal range. |
| research_hypotheses | question, hypothesis, owner, status. |
| agent_runs | hypothesis, graph version, checkpoint, status. |
| source_documents | URI, published_at, hash, extracted metadata. |
| feature_sets | snapshot_id, schema_hash, feature version. |
| experiments | hypothesis_id, configuration, owner. |
| experiment_runs | parameters, metrics, status, artifacts. |
| model_versions | registry URI, version, source run, model hash. |
| strategies | model_version_id, parameters, risk profile. |
| backtest_runs | strategy, snapshot, execution assumptions, metrics. |
| promotion_requests | candidate, checks, approval status. |
| approval_decisions | request, reviewer, decision, reason, timestamp. |
| paper_accounts | provider, external account ID, mode. |
| orders | strategy, client_order_id, state, submitted quantity. |
| execution_fills | order, quantity, price, fees, timestamp. |

| Table | Important fields / responsibility |
|---|---|
| positions | account, instrument, quantity, average price. |
| outcome_evaluations | prediction, realised value, horizon, evaluation status. |
| drift_events | model version, metric, severity, evidence. |
| audit_events | actor, action, target, timestamp, metadata. |
| outbox_events | event type, payload, publish status, retry metadata. |

### Relationship model

```
Research Hypothesis
  -> Agent Runs
  -> Experiments
       -> Data Snapshot
       -> Feature Set
       -> Experiment Runs
            -> Model Versions
                 -> Strategies
                      -> Backtests
                      -> Promotion Requests
                           -> Approval Decisions
                      -> Paper Orders
                           -> Execution Fills
                           -> Positions
                                -> Outcome Evaluations
                                     -> Drift Events / New Research
```

### Database engineering rules

- Do not destructively edit historical experiments; changed inputs create a new version.
- Enforce foreign keys and uniqueness constraints, especially for idempotency keys.
- Keep audit records append-only for application users; corrections should be recorded as new events.
- Use Alembic migrations and test both clean installation and upgrade paths.
- Use a transactional outbox: write the business record and event in one transaction, then publish asynchronously.
- Use indexes for common access patterns such as owner_id, status, created_at, instrument_id and experiment_id.

# 8. AI and research-engine architecture

The AI system should use specialized agents with restricted tools and explicit input/output contracts. Avoid one unrestricted agent with access to all records, files, Python execution and broker functions.

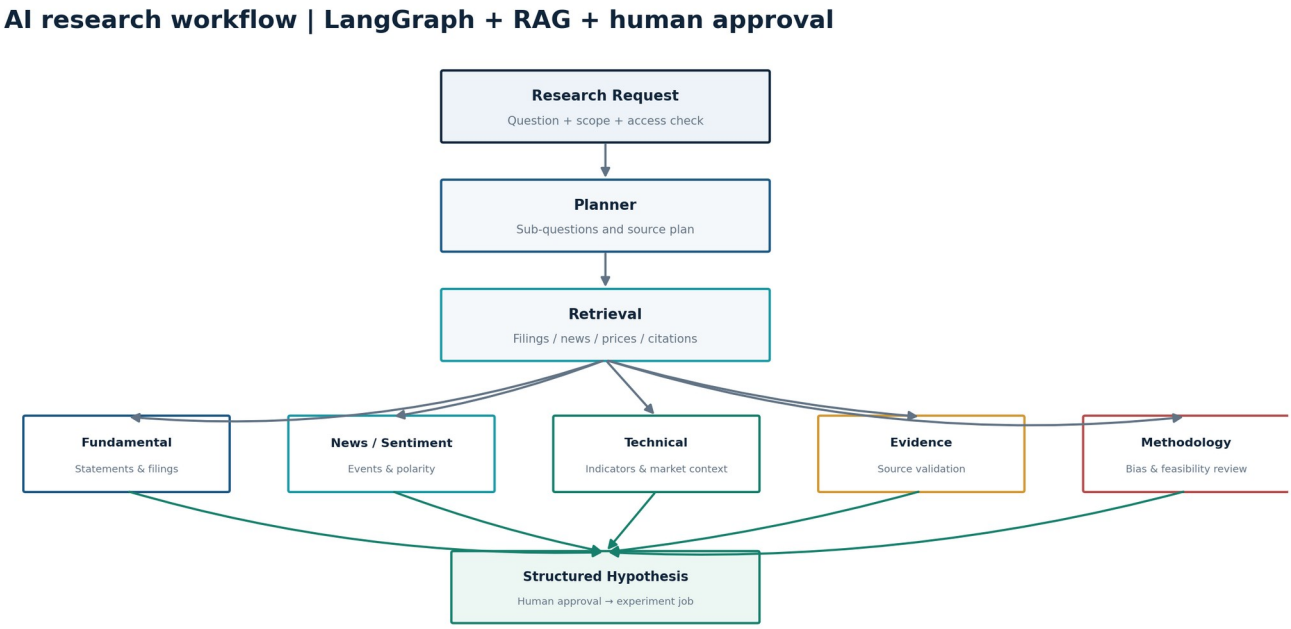



*Figure 3. Research workflow: parallel analyst roles, evidence aggregation, methodology review and human approval.*

## Agent contracts

| Agent | Input | Output |
|---|---|---|
| Planner | Research question, scope and available tools | Sub-questions, source plan and analysis steps. |
| Fundamental Analyst | Filings and financial statements | Structured fundamental findings with citations. |
| Sentiment Analyst | News, filings and event documents | Sentiment, entities, event type and supporting passages. |
| Technical Analyst | Historical market and volume data | Descriptive indicators and market context. |
| Evidence Aggregator | Analyst outputs and source references | Deduplicated claims with traceable evidence. |
| Quant Researcher | Research evidence and constraints | Testable hypothesis, targets and proposed features. |
| Methodology Critic | Proposed experiment | Leakage, bias, feasibility and evaluation review. |
| Final Synthesizer | Verified evidence and review results | Structured research proposal for human approval. |

## LangChain and RAG pipeline

**1.** Ingest an uploaded or permitted public document; validate and hash it.

2. Extract text and preserve source metadata, tables, headings and publication times.
3. Chunk by meaningful sections, not only arbitrary character counts.
4. Generate embeddings and store them with document and chunk metadata.
5. Retrieve with semantic vector search and keyword search.
6. Rerank candidates; retain source location and relevant timestamps.
7. Generate a structured answer with citations to retrieved evidence.
8. Verify claim-to-source links and reject unsupported claims.

Suggested starting tools: LangChain loaders and retrievers, PostgreSQL full-text search, pgvector, a local embedding model, and a lightweight cross-encoder reranker. LLM providers should be optional adapters.

### AI safety controls

- Do not execute arbitrary LLM-generated Python on the application server.
- Do not permit unrestricted filesystem or database access.
- Enforce tenant/resource permissions in the backend, not in prompts.
- Treat uploaded documents and web content as untrusted data.
- Do not let documents override system policies or grant tool permissions.
- Do not allow agents to change risk limits, promote models or place orders directly.
- Persist graph state and require a human approval node before creating promotion or execution requests.

### Evaluation

Create a golden test set of approximately 30-50 research questions initially, then expand. Evaluate retrieval precision/recall, citation correctness, evidence completeness, answer faithfulness, structured-output validity, tool success rate, human-review agreement, latency and cost. RAGAS/DeepEval may help evaluate some aspects, but financial values and calculations must also be checked with deterministic code.

## 9. ML, deep learning and sentiment engineering

The ML layer is an experimentation platform, not a collection of unrelated training scripts. Start with one forecasting target, such as the next five trading days return or a directional class, before expanding to volatility or regime prediction.

| Model | Application | Priority |
|---|---|---|
| Naive baseline | Reference forecast such as zero return or last observation | Essential |
| Logistic Regression | Directional baseline | Essential |
| Random Forest | Nonlinear tabular baseline | High |
| XGBoost | Return or direction modelling | Essential |
| LightGBM | Competing gradient-boosted model | High |
| LSTM | Sequential time-series model | High |
| Transformer | Advanced time-series comparison | Later |
| FinBERT / licensed financial NLP | Financial sentiment classification | High |

## Training workflow

```
Versioned snapshot
  -> feature engineering
  -> target and label construction
  -> chronological train / validation / test split
  -> naive and statistical baselines
  -> XGBoost / LightGBM / LSTM / sentiment models
  -> walk-forward evaluation
  -> MLflow run and artifacts
  -> model registry
  -> candidate review
```

## Feature families

- Returns: log returns, lagged returns and rolling cumulative returns.
- Trend and momentum: moving averages, RSI, MACD and price momentum.
- Volatility: rolling volatility, ATR and range-based features.
- Volume: volume change, relative volume and liquidity proxies.
- Relative features: market-relative and sector-relative returns.
- Text: news sentiment, entities, event type and source confidence.
- Fundamentals: only point-in-time available filing facts and ratios.
- Context: time, market regime and instrument metadata.

## Evaluation

| Task | Metrics |
|---|---|
| Regression | MAE, RMSE, directional accuracy, information coefficient and rank IC. |
| Classification | Precision, recall, F1, ROC-AUC, PR-AUC and probability calibration. |
| Strategy-level | Net return, Sharpe, Sortino, max drawdown, turnover, exposure, costs and benchmark-relative results. |

A high R-squared is not enough. Model promotion requires evidence of useful out-of-sample strategy performance after transaction costs and risk constraints.

## LSTM engineering

Start with a small PyTorch LSTM using a rolling window of engineered features. Fit scalers and feature selection on training data only. Maintain chronological splits and compare against naive and tree-based models. Add a Transformer only when the simpler baseline is stable and the extra complexity can be justified.

### Sentiment output contract

```
{
  "instrument": "MSFT",
  "sentiment": "negative",
  "score": -0.62,
  "event_type": "earnings_guidance",
  "source_document_id": "doc-104",
  "published_at": "2026-10-15T12:00:00Z",
  "available_at": "2026-10-15T12:05:00Z",
  "model_version": "sentiment-v1"
}
```

The example is illustrative. Sentiment scores must come from an actual model and include timestamps and provenance.

## 10. Quantitative backtesting and strategy engineering

Separate research backtesting from execution simulation. Vectorized tools are useful for quick exploration; an event-driven simulator is needed for portfolio accounting and realistic order-state behaviour.

### Two engines

| Engine | Responsibility |
|---|---|
| Research backtest | Fast exploratory strategy calculations, parameter comparison and charts; VectorBT may be used. |
| Execution simulator | Orders, cash, positions, next-bar fills, costs, partial fills, rejections and portfolio-level accounting. |

### Simulation workflow

```
Approved strategy configuration
  -> point-in-time data
  -> signal generation
  -> deterministic risk constraints
  -> order intent
  -> execution simulator
  -> next valid bar / fill model
  -> portfolio accounting
  -> equity curve and trade ledger
  -> benchmark comparison
  -> risk metrics and report
```

### Backtesting rules

- Do not execute on the same bar close if the signal uses that bar close; use next-bar or explicitly modelled execution.
- Include fees, spread and configurable slippage.
- Account for splits and dividends using a documented adjustment policy.
- Use a historically appropriate, frozen universe; where possible retain delisted assets.
- Apply the same benchmark and test windows to candidate and baseline.
- Save the exact dataset snapshot, strategy config, code version and execution assumptions.
- Record unfilled, partial and rejected simulated orders.

- Separate predictive model quality from trading execution quality.
- Prevent future filings, news or fundamentals from entering historical decisions.

### Required metrics

| Category | Metrics |
|---|---|
| Return | Net total return, annualized return, benchmark-relative return. |
| Risk | Volatility, Sharpe, Sortino, max drawdown, downside deviation. |
| Trading | Turnover, trade count, average holding period, exposure, fill rate. |
| Costs | Commission, spread, slippage and estimated market impact. |
| Robustness | Walk-forward windows, seed variation, parameter sensitivity and regime analysis. |

Interview topics: explain look-ahead bias, survivorship bias, selection bias, data leakage, corporate actions, fill assumptions, benchmark selection and why an attractive backtest can fail in paper execution.

## 11. Reinforcement learning portfolio management

RL has one defined responsibility: learn a portfolio allocation policy under explicit risk, liquidity and transaction-cost constraints. It should not replace the research and prediction system.

| Element | Proposed design |
|---|---|
| Environment | Historical portfolio simulator with chronological data access. |
| State | Recent returns, momentum, volatility, cash, current holdings and current weights. |
| Action | Target portfolio weights or allocation adjustments. |
| Reward | Net portfolio return adjusted for risk and costs. |
| Algorithms | PPO first; SAC as a comparison for continuous action spaces. |
| Evaluation | Walk-forward tests, multiple seeds, and equal-weight / buy-and-hold / classical baselines. |

### Illustrative reward

$$r_t = \log(V_t / V_{(t-1)}) - \lambda_1 C_t - \lambda_2 T_t - \lambda_3 D_t$$

V_t is portfolio value; C_t represents transaction cost; T_t represents unnecessary turnover; D_t represents a drawdown penalty; lambda terms are configured weights. This is an initial design proposal, not an optimal reward claim.

### RL pipeline

1. Generate a versioned historical environment.
2. Define and document observation, action and reward spaces.

**3.** Implement costs, cash constraints, concentration limits and position bounds.

**4.** Validate the Gymnasium environment using check_env.

**5.** Train PPO on training windows and tune on validation windows.

**6.** Evaluate on unseen chronological test windows.

**7.** Compare with equal-weight, buy-and-hold and classical allocation.

**8.** Track checkpoints, seeds, configs and metrics in MLflow/TensorBoard.

**9.** Register the policy as a candidate; require the same promotion gate.

**Safety boundary**
An RL policy may propose target weights, but a separate deterministic risk engine validates and clips or rejects allocations before any paper order is created.

# 12. Model governance and promotion

Training completion is not evidence that a model is ready. Model lifecycle transitions must be explicit, recorded and enforceable.

```
TRAINED
  -> REGISTERED
  -> VALIDATION_PENDING
       -> REJECTED
       -> VALIDATION_PASSED
            -> HUMAN_APPROVAL_PENDING
                 -> REJECTED
                 -> APPROVED_FOR_PAPER
                      -> PAPER_MONITORING
                           -> SUSPENDED / RETIRED
```

## Promotion checks

| Check | Required evidence |
|---|---|
| Dataset provenance | Snapshot ID, source manifest and data hash. |
| Model lineage | Training run, code commit, configuration and artifact hash. |
| Temporal validation | No failed leakage or availability-time checks. |
| Baseline comparison | Candidate meets configured improvement criteria. |
| Risk | Drawdown, concentration, exposure and turnover within limits. |
| Robustness | Multiple chronological windows and sensitivity tests. |
| Costs | Net performance after documented transaction costs. |
| Reproducibility | Inputs and configuration can be reconstructed. |
| Human approval | Authorized reviewer and decision rationale recorded. |

### Example experiment manifest

```
{
  "experiment_id": "exp-001",
  "hypothesis_id": "hyp-001",
  "data_snapshot_id": "snapshot-001",
  "feature_schema_version": "v1",
  "target": "forward_5d_return",
  "model_family": "xgboost",
  "model_config_hash": "sha256:...",
  "training_start": "2022-01-01",
  "training_end": "2024-12-31",
  "validation_start": "2025-01-01",
  "validation_end": "2025-06-30",
  "test_start": "2025-07-01",
  "test_end": "2025-12-31",
  "git_commit": "...",
  "random_seed": 42
}
```

Example dates and identifiers are illustrative. MLflow should store run artifacts and lineage; PostgreSQL stores business-level promotion and approval records.

## 13. Paper execution and operational risk

The first release supports local simulation and optionally a broker paper account. Keep provider-specific details behind a common adapter. Do not provide a real-money trading endpoint.

### Broker interface

```
class BrokerAdapter:
    def get_account(self): ...
    def get_positions(self): ...
    def submit_order(self, order): ...
    def get_order(self, order_id): ...
    def cancel_order(self, order_id): ...
    def list_orders(self, status=None): ...
```

| Adapter | Role |
|---|---|
| SimulationBroker | Deterministic local paper execution for development and tests. |
| AlpacaPaperBroker | Optional external paper-trading account adapter. |

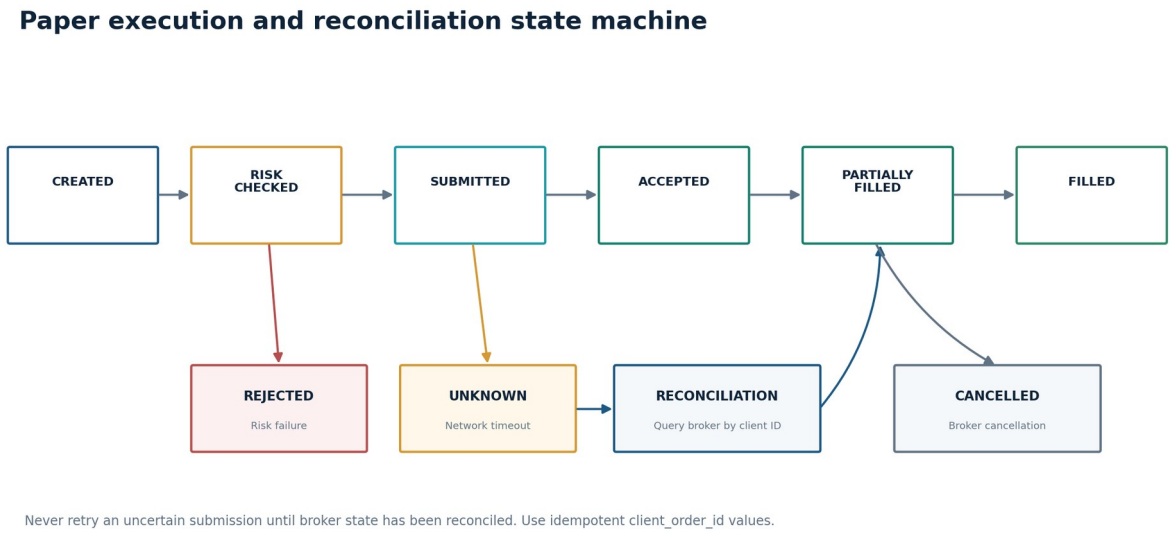



*Figure 4. Example internal order state machine. Provider states must be mapped into controlled internal states.*

## Required safety rules

- Unique, idempotent client order IDs.
- Duplicate-order detection.
- Position-size and portfolio-exposure limits.
- Maximum daily simulated loss and drawdown thresholds.
- Stale-market-data rejection.
- Order and market-hours validation.
- Application-level kill switch and per-strategy disable control.
- Periodic broker-to-ledger reconciliation.
- Manual approval before enabling a paper strategy.

Never assume a timed-out broker request failed. Query the broker by client order ID and reconcile before any retry. Record decision price, expected fill, actual fill, quantity, fees, slippage, realised P&L, strategy ID and model version for each order/fill.

# 14. APIs and application contracts

Expose versioned REST endpoints under /api/v1. Use Pydantic request/response models, consistent error envelopes, pagination, authentication dependencies, request IDs and idempotency keys for side-effecting operations.

| API area | Example endpoints |
|---|---|
| Authentication | POST /auth/login; POST /auth/refresh |
| Users and roles | GET /users/me; GET /admin/users |
| Instruments | GET /instruments; GET /instruments/{symbol} |
| Data snapshots | POST /data/ingestions; GET /data/snapshots/{id} |
| Research | POST /research/hypotheses; GET /research/hypotheses/{id} |
| Research jobs | POST /research/runs; GET /jobs/{id} |
| Documents | POST /documents; GET /documents/{id} |

| API area | Example endpoints |
|---|---|
| Experiments | POST /experiments; GET /experiments/{id} |
| Training | POST /experiments/{id}/runs |
| Models | GET /models; GET /models/{id}/versions |
| Promotion | POST /models/{id}/promotion-requests |
| Backtesting | POST /backtests; GET /backtests/{id} |
| Strategies | POST /strategies; GET /strategies/{id} |
| Approvals | POST /approvals/{id}/decisions |
| Paper trading | POST /paper/orders; GET /paper/orders |
| Portfolio | GET /paper/positions; GET /paper/performance |
| Feedback | GET /feedback/outcomes; GET /feedback/drift-events |
| Audit and health | GET /audit/events; GET /health/live; GET /health/ready |

## Asynchronous job contract

```
{
  "job_id": "job-001",
  "status": "QUEUED",
  "resource_type": "BACKTEST",
  "resource_id": "backtest-001",
  "created_at": "2026-11-02T10:00:00Z"
}
```

The timestamp and IDs above are examples. A client polls GET /api/v1/jobs/{job_id} or subscribes to a progress channel. Do not keep an HTTP request open while a model trains or an agent performs long research.

## Job state

```
QUEUED -> RUNNING -> SUCCEEDED
                  -> RETRYING -> RUNNING
                  -> FAILED
                  -> CANCEL_REQUESTED -> CANCELLED
```

Background tasks can be delivered more than once. Each task must check its business record, idempotency key and expected state before writing results or causing side effects.

# 15. Application folder structure

```
finsight-researchops/
|-- apps/
|   |-- api/
|   |   |-- main.py
|   |   |-- dependencies.py
|   |   `-- routes/
|   |-- worker/
|   |   |-- celery_app.py
|   |   `-- tasks/
|   `-- web/
|       `-- src/ (pages, components, features, api)
|-- src/finsight/
|   |-- core/ (config, logging, security)
|   |-- identity/
|   |-- market_data/ (providers, ingestion, contracts)
|   |-- documents/ (parsers, chunking, embeddings, retrieval)
|   |-- research/ (hypotheses, agents, graphs, evaluations)
|   |-- features/
|   |-- experiments/
|   |-- models/ (baselines, xgboost, pytorch, sentiment)
|   |-- reinforcement_learning/ (environments, rewards, policies)
|   |-- backtesting/
|   |-- risk/
|   |-- governance/
|   |-- execution/
|   |-- feedback/
|   |-- analytics/
|   `-- infrastructure/ (database, repositories, storage, messaging)
|-- migrations/
|-- configs/ (development, staging, production)
|-- sql/ (schema, analytics)
|-- tests/ (unit, integration, contract, security, temporal_integrity, e2e, recovery)
|-- infra/ (docker, monitoring, deployment)
|-- docs/ (architecture, api, decisions, threat_model, operations, methodology)
|-- scripts/
|-- .github/workflows/
|-- .env.example
|-- docker-compose.yml
|-- Dockerfile
|-- Makefile
|-- pyproject.toml
|-- README.md
`-- LICENSE
```

## Code organization rules

- API routes call application services; do not mix HTTP concerns with domain logic.
- Domain code must not import FastAPI, Celery or frontend packages.
- Training code must not depend on UI components.
- Broker adapters implement a shared interface and have contract tests.
- Prompts, feature definitions, strategies and model configs are versioned artifacts.
- Long-running operations produce job IDs and trace IDs.
- Each important domain operation has an audit event.

# 16. Security and permissions

| Role | Permissions |
|---|---|
| Admin | User, role and platform configuration. |

| Role | Permissions |
|---|---|
| Research Analyst | Documents, research and hypothesis creation. |
| Quant Researcher | Features, experiments, training and backtests. |
| Risk Reviewer | Risk review, promotion and suspension decisions. |
| Portfolio Analyst | Paper positions, fills and performance. |
| Read-only Viewer | Approved reports and permitted dashboards. |

### Required security controls

- Authentication with short-lived access tokens and secure refresh/revocation.
- Resource-level authorization in backend queries; random IDs are not a security boundary.
- Secrets in a secret manager or local environment; never commit credentials.
- File upload validation for size, MIME type, extension, content and storage path.
- LLM prompt-injection defence: documents are untrusted; tools have narrow permissions.
- Append-only audit trail for model promotion, access changes, approvals, configuration changes and order actions.
- Rate limits, per-user job quotas, document-size caps and concurrency limits.
- Database least-privilege roles, TLS, backup encryption and controlled migration permissions.
- Dependency and container vulnerability scanning.

### Risk controls

- Global paper-trading disable switch.
- Per-strategy enable/disable control.
- Maximum permitted position size and total exposure.
- Maximum daily loss and portfolio drawdown thresholds.
- Stale data checks and provider-health checks.
- Separate permission to approve paper execution.
- Audit record for manual overrides.

The AI model must never change these operational controls. Review OWASP API Security guidance, especially object-level authorization, broken authentication, unrestricted resource consumption and unsafe API consumption.

## 17. Testing and quality strategy

Testing is part of the system architecture. It should be possible to prove temporal correctness, job recovery, authorization, model lineage and execution safety.

| Test layer | What it verifies |
|---|---|
| Unit | Indicators, features, reward logic, transaction costs and risk calculations. |
| Integration | PostgreSQL, Redis, object storage, MLflow and worker interactions. |
| Contract | Market-data, LLM and broker adapter schemas and error behavior. |
| Temporal integrity | No future information in snapshots, features or decisions. |

| Test layer | What it verifies |
|---|---|
| API security | Authentication, resource ownership, invalid inputs and rate limits. |
| End-to-end | Full hypothesis-to-paper-order user journey. |
| Recovery | Worker interruption, retry and duplicate handling. |
| Load | Request throughput, queue delay, worker concurrency and resource caps. |
| Reproducibility | Same inputs/config/seed yields equivalent results within defined tolerance. |

## Five critical automated tests

**Look-ahead protection:** A feature with available_at after decision_time is rejected by dataset validation.

**Duplicate background jobs:** Submitting the same task twice with one idempotency key does not create duplicate experiment results.

**Model promotion protection:** A candidate with missing provenance or failed risk evaluation cannot be promoted.

**Worker recovery:** Terminate a worker mid-run, restart it and verify safe recovery without duplicate artifacts or results.

**Paper-order timeout:** Simulate broker timeout after submission; the worker reconciles broker state before any retry.

## Operational measures

| Area | Metrics |
|---|---|
| API | p50/p95 latency, error rate and request count. |
| Workers | Queue delay, task duration, retries and stuck jobs. |
| AI agents | Tool success, citation coverage, workflow completion and token cost. |
| RAG | Retrieval precision/recall, relevance and faithfulness. |
| ML | Model metrics, calibration, feature drift and data freshness. |
| RL | Evaluation reward, drawdown, turnover and benchmark performance. |
| Backtesting | Runtime, fill ratio, trade count, costs and drawdown. |
| Execution | Order rejects, reconciliation mismatches and slippage. |
| Infrastructure | CPU, memory, disk, database connections and object-store health. |

Set service-level objectives only after measuring realistic staging load. Do not claim a specific uptime or throughput without evidence.

# 18. CI/CD and deployment architecture

A code change should pass automated quality gates before it reaches a staging deployment. Model artifacts and application images should be versioned separately.

```
Pull request
 -> formatting / lint / type checks
 -> unit tests
 -> integration tests
 -> security and dependency scans
 -> Docker image build
 -> container tests
 -> staging deployment
 -> migration check
 -> API / UI smoke tests
 -> manual release approval
 -> production deployment
 -> health / monitoring checks
```

## CI tools

- pytest - unit, integration and end-to-end tests.
- ruff - linting and formatting.
- mypy - static type checks.
- pip-audit - Python dependency vulnerability review.
- Bandit - Python security-oriented static analysis.
- Trivy - container image and filesystem vulnerability scanning.
- GitHub Actions - continuous integration and deployment workflows.

## Local Docker Compose services

```
frontend | api | worker | scheduler | postgres | redis | minio | mlflow | prometheus | grafana
```

Use Compose profiles to keep optional services disabled by default. Pin dependency versions and run compatibility tests before upgrading AI or data libraries.

## Example cloud deployment

| Concern | Potential AWS service |
|---|---|
| Frontend | S3 + CloudFront or container hosting. |
| API | ECS Fargate. |
| Background workers | ECS worker tasks. |
| Database | RDS PostgreSQL. |
| Queue/cache | ElastiCache Redis. |
| Artifact storage | S3. |
| Secrets | Secrets Manager. |
| TLS and routing | Application Load Balancer. |

| Concern | Potential AWS service |
|---|---|
| Logs and alerts | CloudWatch and OpenTelemetry collector. |

Kubernetes is not a version-one requirement. A well-operated container deployment with migrations, backups, monitoring and rollback is preferable to introducing orchestration without a measured need.

# 19. Frontend pages and working application flow

The frontend should function as an analyst research workstation, not as a generic chatbot. The dashboard should show job state, lineage, approvals, evidence and operational health.

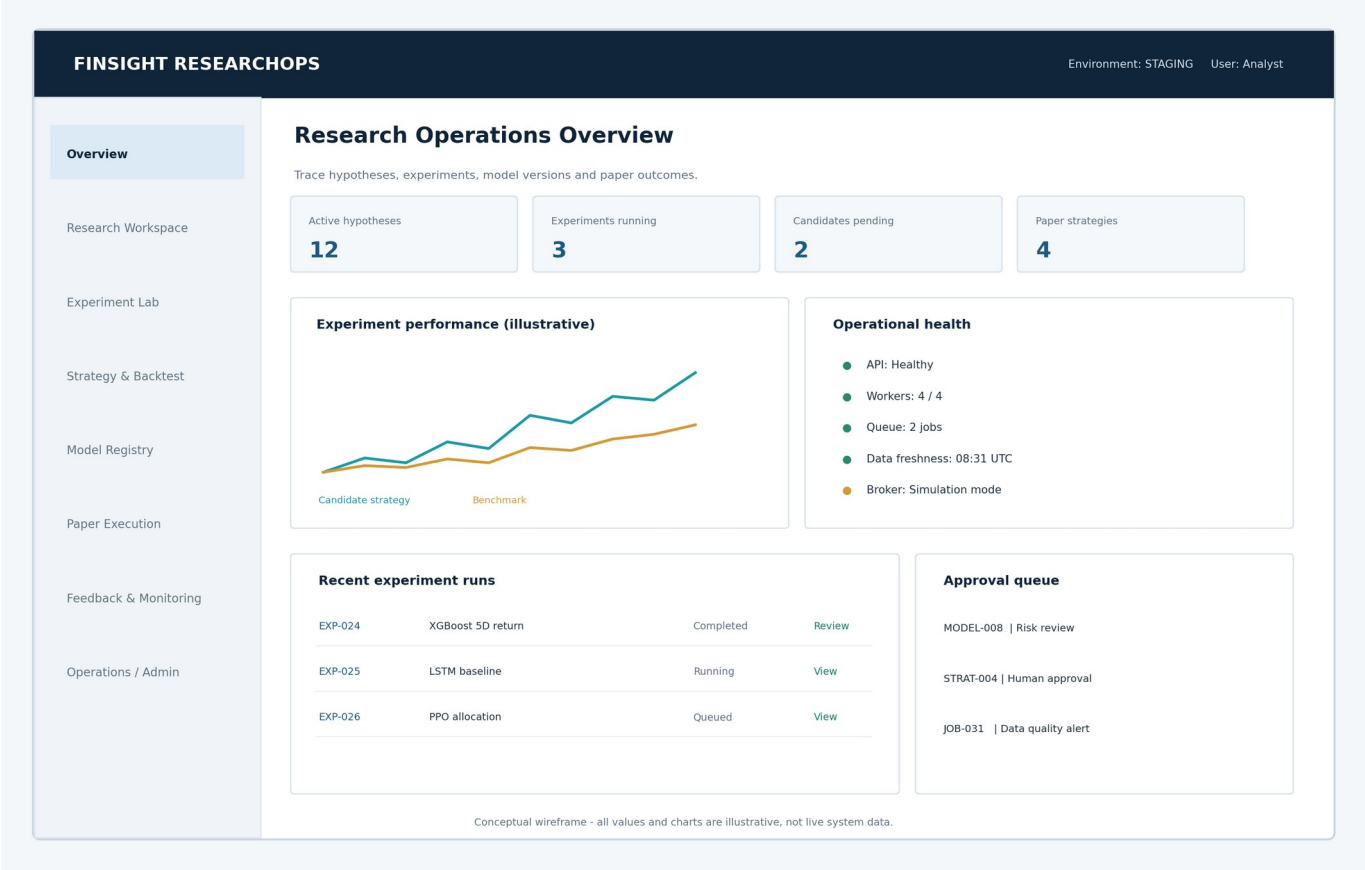



*Figure 5. Conceptual frontend overview wireframe. All displayed numbers and charts are illustrative, not live data.*

## Required frontend pages

| Page | Key functions |
|---|---|
| 1. ResearchOps Overview | Active hypotheses, experiments, candidate models, alerts, job health and paper performance. |
| 2. Research Workspace | Company search, filings, source viewer, cited AI conversation, notes and hypothesis creation. |
| 3. Experiment Lab | Dataset, target, features, model family, run parameters, progress, metrics and comparisons. |

| Page | Key functions |
|---|---|
| 4. Strategy & Backtest Lab | Strategy configuration, benchmark comparison, equity curves, trade history, costs and risk reports. |
| 5. Model Registry & Governance | Model versions, lineage, evaluation, approval requests, promotion history and rollback. |
| 6. Paper Execution | Approved strategies, orders, fills, cash, positions, exposure, rejects and reconciliation. |
| 7. Feedback & Monitoring | Forecast errors, drift, expected vs realised outcomes, degradation and proposed experiments. |
| 8. Operations & Administration | Users, roles, audit logs, worker health, queue status, provider status and metrics. |

## Primary end-to-end user journey

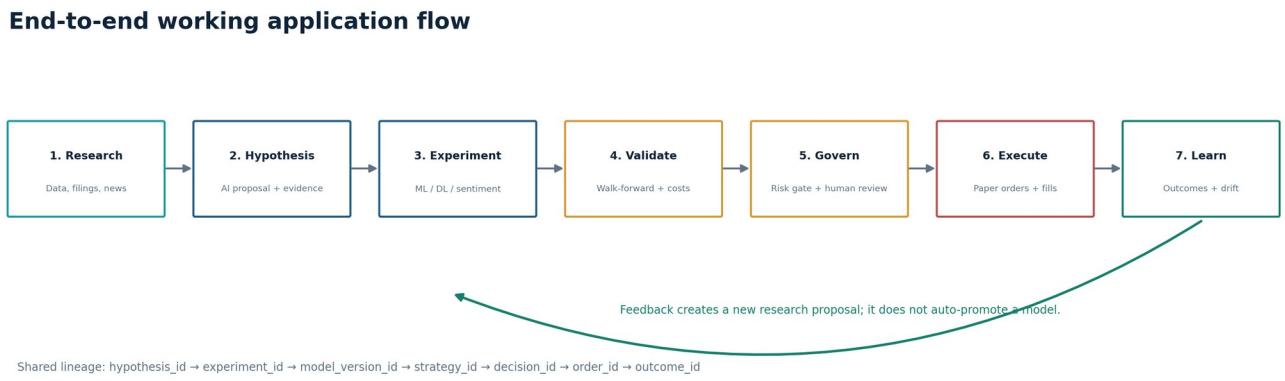



*Figure 6. User journey from research to measured outcomes.*

**1.** Researcher logs in and chooses an instrument or company.
**2.** They inspect permitted filings, news and available market data.
**3.** They ask the research team to formulate a testable hypothesis.
**4.** The system generates a report with source citations and a proposed experiment.
**5.** Researcher approves the hypothesis.
**6.** Experiment manager creates a versioned dataset and training job.
**7.** ML worker trains baseline, XGBoost and LSTM candidates.
**8.** Quant simulator runs walk-forward backtests.
**9.** Risk reviewer evaluates performance against the benchmark and risk limits.
**10.** Candidate model is registered and submitted for promotion approval.
**11.** Approved strategy runs in paper mode only.
**12.** Order ledger records submission, state transitions, fills and positions.
**13.** Feedback module compares forecasts, intended fills and realised outcomes.
**14.** Research agent proposes the next experiment based on measured evidence.

### Identifiers shared across pages

The application should display and filter by hypothesis_id, experiment_id, model_version_id, strategy_id, decision_id, order_id and outcome_id. Users should be able to navigate from a research hypothesis to all downstream artifacts and outcomes.

# 20. Implementation roadmap and acceptance criteria

Build the product in vertical slices. Each phase must pass its quality gate before the next phase is treated as complete.

| Phase / deliverable | Build scope | Acceptance criteria |
|---|---|---|
| 0 - Repository and engineering foundation | Python packaging, React/TypeScript, Docker, PostgreSQL, Redis, migrations, logging and architecture decision records. | Clean clone starts dev stack; migrations work on blank DB; CI runs lint and basic tests; secrets excluded. |
| 1 - Identity, catalogue and ingestion | Login/RBAC, instruments, providers, SEC ingestion, historical market data, Parquet snapshots and data checks. | Snapshots have source, manifest and hash; retries are safe; duplicates are prevented; stale data is visible. |
| 2 - Experiment and strategy foundation | Hypotheses, experiment config, feature definitions, baseline model, strategy registry and reproducible jobs. | Researcher can create hypothesis and experiment; outputs link to exact dataset and configuration. |
| 3 - ML and deep learning | Baseline, XGBoost, LSTM, sentiment model, MLflow tracking and artifacts. | Models use same approved chronological split; baseline comparison exists; rerun is equivalent within tolerance. |
| 4 - Financial research AI | LangChain RAG, LangGraph analysts, evidence verification, proposal output and checkpoint persistence. | Citations refer to sources; failed graph resumes; agents cannot bypass risk or execution. |
| 5 - Backtesting and governance | Portfolio accounting, next-bar fills, costs, walk-forward tests, benchmarks and promotion rules. | Leakage tests pass; candidate and benchmark use identical windows; risk failures block promotion. |
| 6 - Reinforcement learning | Gymnasium environment, reward config, PPO training, SAC comparison and TensorBoard/MLflow. | Environment checker passes; out-of-sample evaluation exists; risk constraints enforced. |
| 7 - Paper execution | Simulation broker, optional paper broker, order state machine, idempotency, fills and kill switch. | No live trading route; duplicate requests blocked; timeout reconciliation tested. |
| 8 - Outcome feedback | Prediction/outcome matching, drift, strategy degradation and next-experiment suggestions. | Matured forecasts link to outcomes; feedback proposes experiments but never auto-replaces models. |
| 9 - Deployment and operations | Staging/public deployment, security scans, tracing, metrics, backup/restore, load and recovery tests. | Deployment repeatable; worker interruption recoverable; restore demonstrated; rollback documented. |

# 21. Project configuration and developer experience

A professional project should be straightforward for another developer to install, test, configure and contribute to.

## Example .env.example

```
APP_ENV=development
APP_NAME=FinSight-ResearchOps
DATABASE_URL=postgresql+psycopg://user:password@localhost:5432/finsight
REDIS_URL=redis://localhost:6379/0
OBJECT_STORAGE_ENDPOINT=http://localhost:9000
OBJECT_STORAGE_BUCKET=finsight-artifacts
MLFLOW_TRACKING_URI=http://localhost:5000
LLM_PROVIDER=mock
LLM_API_KEY=
EMBEDDING_PROVIDER=local
MARKET_DATA_PROVIDER=demo
MARKET_DATA_API_KEY=
SEC_EDGAR_USER_AGENT=FinSightResearchOps contact@example.com
BROKER_MODE=simulation
BROKER_API_KEY=
BROKER_SECRET_KEY=
PAPER_TRADING_ENABLED=false
LIVE_TRADING_ENABLED=false
JWT_SECRET=
LOG_LEVEL=INFO
```

This is a template only. Actual variable names must match the settings implementation. Never store actual keys in the example file.

## Developer commands

```
make install
make dev
make test
make test-integration
make lint
make typecheck
make migrate
make seed-demo-data
make train-baseline
make run-backtest
make worker
make monitoring
make docker-build
```

## Required documentation

| File | Content |
|---|---|
| README.md | Overview, prerequisites, install, demo and quickstart. |
| SYSTEM_ARCHITECTURE.md | Components, deployment and data flow. |
| DATABASE_DESIGN.md | Entities, relationships, indexes and migrations. |
| API.md | Endpoints, authentication, schemas and error model. |
| AI_AGENT_DESIGN.md | Graph nodes, tools, prompts, state and safety. |
| ML_METHODOLOGY.md | Features, labels, splits, metrics and baselines. |
| RL_METHODOLOGY.md | Environment, state, actions, rewards and benchmarks. |
| BACKTESTING_METHODOLOGY.md | Fill model, costs, benchmarks and bias controls. |

| File | Content |
|---|---|
| SECURITY.md | Threat model, permissions and secret handling. |
| DEPLOYMENT.md | Local, staging and cloud deployment. |
| OPERATIONS_RUNBOOK.md | Alerts, recovery, rollback and incident steps. |
| SOURCES_AND_ATTRIBUTION.md | Repository references and licence notes. |

# 22. Production-readiness checklist

This checklist is an acceptance tool. Items are not considered complete because a page exists; they require evidence from code, tests and a running environment.

## Application

- ☐ Frontend and backend are deployed separately.
- ☐ API is versioned and documented.
- ☐ Database migrations are automated and tested.

## Security

- ☐ Authentication and RBAC work.
- ☐ Object-level permissions are tested.
- ☐ Secrets are not stored in Git.
- ☐ Upload and LLM inputs are validated.

## Data

- ☐ Source provenance is preserved.
- ☐ Data snapshots are immutable.
- ☐ Temporal-integrity tests pass.

## AI and ML

- ☐ Agent state persists across interruptions.
- ☐ Model runs are reproducible.
- ☐ Model versions have lineage and approval status.

## Quant and RL

- ☐ Backtests include costs and next-bar execution.
- ☐ Benchmarks and walk-forward tests exist.
- ☐ RL policies obey deterministic risk limits.

## Execution

- ☐ Paper broker is the only enabled broker mode.
- ☐ Duplicate order submissions are prevented.
- ☐ Timeout reconciliation is tested.

### Operations

- ☐ Background jobs recover safely.
- ☐ Logs, traces and metrics are available.
- ☐ Backup and restore are demonstrated.
- ☐ CI/CD and rollback are working.

**Readiness statement**
Only describe a feature as production-engineered after the corresponding acceptance test has passed and the evidence can be demonstrated.

# 23. Interview demonstration plan

Prepare six deliberate demonstrations. The most important is not a prediction chart but an end-to-end trace across the platform.

| Demo | What to demonstrate |
|---|---|
| 1. Research traceability | Create a hypothesis; show source documents, citations, agent graph, structured proposal and approval. |
| 2. Reproducible experiment | Show dataset hash, feature schema, Git commit, config and model version; rerun and compare outputs. |
| 3. Model comparison | Compare baseline, XGBoost and LSTM; explain why a model was accepted or rejected. |
| 4. Quantitative risk | Run walk-forward backtest; show fees, slippage, drawdown, turnover and benchmark-relative performance. |
| 5. Failure recovery | Interrupt a worker, restart it and demonstrate safe recovery without duplicate results. |
| 6. Outcome feedback | Show paper trade, fill reconciliation, predicted-versus-realised result and a new research proposal. |

Keep a short demo dataset and stable reproducible scenario. Explain the trade-offs and known limitations instead of claiming guaranteed profitability or enterprise usage.

# 24. Definition of success

| Deliverable | Evidence expected |
|---|---|
| Full-stack platform | Deployable frontend and API with documented setup. |
| Software architecture | Diagrams, API contracts and architecture decision records. |
| Generative AI | Persistent LangGraph workflows and grounded RAG. |
| ML/DL | Reproducible model runs and comparison reports. |
| RL | PPO/SAC evaluation against allocation baselines. |

| Deliverable | Evidence expected |
|---|---|
| Quant research | Point-in-time snapshots and realistic backtests. |
| Model governance | Registry, validation and approval workflow. |
| Paper execution | Order state machine, idempotency and reconciliation. |
| Feedback loop | Predicted-versus-realised outcome reports. |
| Engineering | Automated tests, CI/CD, security and monitoring. |
| Operations | Recovery, backup/restore and load-test evidence. |

The project is successful when an external reviewer can clone or deploy it, understand its architecture, run the documented workflow, inspect the evidence and reproduce a representative experiment.

# 25. Resume positioning

Use this resume entry only after the corresponding capabilities have actually been implemented, tested and demonstrated.

**Proposed resume title**
FinSight ResearchOps - AI-Powered Financial Research & Quantitative Strategy Operations Platform

Technologies: Python, FastAPI, React, TypeScript, PostgreSQL, Redis, Celery, LangChain, LangGraph, RAG, XGBoost, PyTorch, LSTM, Gymnasium, PPO, SAC, MLflow, Docker and OpenTelemetry.

- Engineered a research-to-execution lifecycle connecting financial document intelligence, AI-assisted hypothesis generation, reproducible ML/DL experiments and quantitative strategy evaluation.
- Designed a point-in-time financial data pipeline with immutable dataset snapshots, feature-schema versioning and temporal-integrity checks.
- Developed stateful LangGraph research workflows with evidence retrieval, structured outputs, persistent checkpoints and human approval gates.
- Implemented portfolio backtesting and reinforcement-learning allocation experiments with transaction-cost, risk and benchmark evaluation.
- Built model governance, paper-order reconciliation and prediction-outcome feedback workflows with audit trails and operational monitoring.
- Established automated testing, CI/CD, Docker deployment, background-job recovery and observability.

Only retain bullets supported by code, tests and a demonstration. Do not claim profitability, production banking deployment or unmeasured performance figures.

# 26. Final implementation recommendation

The strongest way to build this product is to establish data lineage, experiment registry, backtesting and risk controls before introducing the most sophisticated AI components.

**Recommended build order**
1. Data lineage and experiment registry. 2. Backtesting and risk engine. 3. ML/DL and model registry. 4. LangGraph financial research. 5. RL portfolio simulation. 6. Paper execution and outcome feedback.

The core innovation is the Research-to-Outcome Ledger, not the number of AI frameworks. The application should show where a hypothesis came from, what dataset and model tested it, why it was approved, how its paper order executed and what the measured outcome was.

Start with the four primary repository references: Quant Strategy Framework, RL Portfolio Management, Financial Research Agent and Stock Agent System. Add the Banking Fraud & Credit Risk ML repository only for selected MLOps patterns. Build the shared domain layer independently and verify all upstream licences before copying code.

Use the first release as a paper-only research and strategy-operations product. Treat production readiness as a set of demonstrated security, reliability, reproducibility, recovery and observability controls—not as a label.

# Repository and technical references

GitHub repositories listed in this plan are independent projects. Their authors retain ownership. Read current README, licence, release notes, dependencies and data terms before reuse.

**Quant Strategy Framework:** https://github.com/niharmaddina/quant-strategy-framework (see Section 3 for reuse notes)
**RL Portfolio Management:** https://github.com/MustafaBeratYavas/rl-based-algorithmic-trading-and-portfolio-management (see Section 3 for reuse notes)
**Financial Research Agent:** https://github.com/Harshithaaramesh/financial-research-agent (see Section 3 for reuse notes)
**Stock Agent System:** https://github.com/mustafagoktugibolar/stock-agent-system (see Section 3 for reuse notes)
**Banking Fraud & Credit Risk ML:** https://github.com/Speaking-Data-NK/banking-fraud-risk-ml (see Section 3 for reuse notes)
**AI/ML Stock Market Prediction:** https://github.com/AISHDM/stock-market-prediction (see Section 3 for reuse notes)
**RLTradingAgent:** https://github.com/maksimprivalov/RLTradingAgent (see Section 3 for reuse notes)
**Machine Learning Alpha Model:** https://github.com/joel-cdev/machine-learning-alpha-model (see Section 3 for reuse notes)
**Microsoft Qlib:** https://github.com/microsoft/qlib (see Section 3 for reuse notes)
**Microsoft RD-Agent:** https://github.com/microsoft/RD-Agent (see Section 3 for reuse notes)
**FinRL-X:** https://github.com/AI4Finance-Foundation/FinRL-Trading (see Section 3 for reuse notes)
**FinRobot:** https://github.com/AI4Finance-Foundation/FinRobot (see Section 3 for reuse notes)
**FinGPT:** https://github.com/AI4Finance-Foundation/FinGPT (see Section 3 for reuse notes)
**TradingAgents:** https://github.com/TauricResearch/TradingAgents (see Section 3 for reuse notes)
**OpenBB:** https://github.com/openbq-org/OpenBB (see Section 3 for reuse notes)
**DocuMind Agentic RAG:** https://github.com/Vishal93727/documind-agentic-rag (see Section 3 for reuse notes)

## Technical documentation

- LangGraph persistence: https://langchain-ai.github.io/langgraph/concepts/persistence/
- LangGraph interrupts / human-in-the-loop: https://langchain-ai.github.io/langgraph/concepts/human_in_the_loop/
- Celery task reliability: https://docs.celeryq.dev/en/stable/userguide/tasks.html
- MLflow Model Registry: https://mlflow.org/docs/latest/model-registry/
- Stable-Baselines3 custom environments: https://stable-baselines3.readthedocs.io/en/master/guide/custom_env.html
- OpenTelemetry Python: https://opentelemetry.io/docs/languages/python/
- OWASP API Security Top 10: https://owasp.org/www-project-api-security/
- SEC EDGAR APIs: https://www.sec.gov/edgar/sec-api-documentation